

Assignment 01 — Observability in the Wild

Adding OpenTelemetry to nopCommerce

Objective

Real software architects rarely build from scratch. More often, you inherit a codebase, need to understand how it works, and make targeted improvements without breaking what's already there.

In this assignment you will instrument **nopCommerce** — a real, actively maintained open-source e-commerce platform — with OpenTelemetry tracing and metrics. You will also critique its existing architecture from an observability standpoint and justify every instrumentation decision you make.

The Project

nopCommerce is a production-grade ASP.NET Core e-commerce platform with over 9,000 GitHub stars and active commercial usage worldwide.

Repository	https://github.com/nopSolutions/nopCommerce
Language	C# / ASP.NET Core
Architecture	Layered — Presentation, Services, Data, Core
Communication	In-process events via IPublisher
OTel out of the box	None

Fork the repository before making any changes.

Your Tasks

1. Read Before You Touch — Architecture Analysis

Before writing a single line of instrumentation code, spend time understanding the codebase. Your documentation must answer:

- How are the layers organised and what are the dependency rules between them?
- How does nopCommerce handle events internally — what is `IEventPublisher` and how is it used?
- Where does the code make it **easy** to add observability, and where does it make it **hard**?
- What would you need to change structurally to instrument it properly — and is that change worth making?

This section is not a summary of the README. It is your architectural reading of the system.

2. Pick One User Flow and Instrument It End-to-End

Choose **one** of the following flows:

Flow	Services Involved
Customer places an order	Basket · Order · Payment · Inventory
Customer searches and views a product	Catalogue · Search · Pricing
Admin publishes a new product	Admin UI · Catalogue · Cache

Your instrumentation must cover:

- **Distributed tracing** — spans from HTTP entry point through service calls to database
- **At least two custom metrics** that provide genuine operational insight (not just request count — that's already available)
- **Sensitive data exclusion** — no emails, payment details or PII in traces or logs

The metrics you choose must be justified. *"I added this because it was easy"* is not a justification. *"This metric would tell an operator that the checkout pipeline is degrading before users start seeing errors"* is.

3. Grafana Dashboard

Set up a Grafana dashboard that visualises your instrumented flow. It must include:

- Trace view for the selected flow (via Jaeger or Tempo)
- At least one panel per custom metric
- A panel showing error rate for the flow

The dashboard should tell a story — someone who has never seen your code should be able to look at it and understand what is happening in the system.

4. Load Test

Provide a load test script (k6, Locust, or JMeter) that drives your selected flow under load. Run it and include screenshots showing your metrics and traces responding to the load in Grafana.

The load test does not need to be sophisticated — it needs to generate enough signal to make your dashboard meaningful.

5. Critique

Write a short section (half a page to one page) addressing:

- What in nopCommerce's design helped or hindered your instrumentation work?
- If you were making architectural decisions on this project going forward, what would you change to make it more observable — and at what cost?
- Where did you have to make a surgical change to the existing code? Why was it necessary, and how did you minimise the impact?

This is the architect's voice. It matters as much as the code.

Deliverables

Deliverable	Details
GitHub repository link	A fork of nopCommerce under your own GitHub account. This is your primary submission — everything else lives here

Deliverable	Details
README.md	How to build, run, and view the dashboard. Must include architecture diagram of your instrumented flow
Grafana dashboard	Exported JSON + screenshots
Load test script	With instructions to run it
Critique	In the README or as a separate CRITIQUE.md
Presentation / Demo	10 minutes — live demo of the dashboard under load + walkthrough of your decisions

Repository Requirements

Your submitted repository must:

- Be a **shared fork with the professor** of **nopSolutions/nopCommerce** — not a copy or a new repo
- Show a clear **commit history** that reflects your work incrementally — a single large commit at the end is not acceptable
- Have all instrumentation, configuration, dashboard exports and load test scripts committed to the repo
- Include a **CRITIQUE.md** at the root level

Submit your repository link via **elearning** before the deadline. The link must point to your fork, not a branch or a pull request.

A repository with no commit history, or one where all changes appear in a single commit, will be treated as incomplete regardless of the quality of the code.

Evaluation Criteria

Area	What We're Looking For
Architectural understanding	Does the analysis show genuine reading of the codebase, or is it superficial?

Area	What We're Looking For
Instrumentation quality	Are spans meaningful and correctly scoped? Is there noise? Is sensitive data excluded?
Metric justification	Are the chosen metrics operationally useful? Could an on-call engineer act on them?
Surgical approach	Are changes minimal and well-placed, or did the student refactor things they didn't need to?
Dashboard clarity	Does the dashboard communicate what is happening without needing explanation?
Critique depth	Does the critique show architectural thinking, or is it just a list of observations?

Hints

On reading the codebase: Start with `Nop.Services` and `Nop.Core`. The service layer is where the business logic lives and where the most meaningful instrumentation points are. `IEventPublisher` is worth understanding early — it is nopCommerce's internal event mechanism and a natural instrumentation boundary.

On choosing metrics: Ask yourself: *if this metric spiked or dropped at 2am, would an engineer know what to do?* If yes, it is a good metric. Request duration and error rate are the baseline — your custom metrics should go beyond that.

On sensitive data: nopCommerce handles real customer data in its service layer. Before you instrument anything in `Nop.Services.Orders` or `Nop.Services.Customers`, check what objects are being passed around. An OTel processor that sanitises span attributes before export is cleaner than trying to remember which fields to exclude everywhere.

On surgical changes: You will likely need to add `ActivitySource` in one or two places where built-in instrumentation does not reach. Keep these changes as close to the infrastructure boundary as possible — not in business logic. If you find yourself modifying a service class significantly, step back and find a better instrumentation point.

On the demo: Run your load test live during the presentation. A dashboard with real traffic tells a better story than screenshots. Have a second terminal ready showing traces in Jaeger or Tempo while the load is running.

Deadline

Presentation: 24th and 25th March